EE5203 L02 Lab Guide

Process eight sensor values with decisions and loops - Basri Erdogan

Mission: Copy your working L01 project and process eight simulated sensor values in one `main.c` file. Clamp invalid values, calculate an average, count high values, and select a numeric status. Prove every result in the debugger.

Prepare before class

- ☐ Bring your working L01 Nucleo-F401RE project and USB data cable.
- ☐ Review assignment, arithmetic, `if/else`, `for`, and array indices.
- ☐ Complete the entry ticket below.

```
int x = 3;
x = x + 2;
x++;
```

1. What is the final value of `x`?
2. Which symbol compares two values: `=` or `==`?
3. What are the valid indices of `int values[4]`?

Learning objectives

By checkoff time, you can:

1. trace assignments and arithmetic statements in order;
2. use comparisons in `if/else` decisions;
3. trace a `for` loop from its first to last valid counter value;
4. process every element of an eight-element array;
5. use debugger evidence to justify a prediction.

Hardware and files

Item	Required value
Board	Nucleo-F401RE
Tools	STM32CubeMX + VS Code
Starting point	Working copy of the L01 project
Project name	`EE5203_L02_<studentnumber>`
File you edit	Generated `Core/Src/main.c`
New source/header files	None
Input	Eight fixed `uint16_t` values
Evidence	Build console, breakpoint, Watch/Expressions

Keep all edits to generated `main.c` inside the appropriate USER CODE regions.

Rules

The eight fixed input values are declared in Checkpoint B. Rules:

- valid sensor values are `0` through `4095`;
- a value above `4095` must become `4095`;
- a high sample is strictly greater than `2500`;
- status code is `-1` below `1500`, `1` above `3000`, otherwise `0`.

Warm-up (first 15 minutes)

Before editing, trace this loop on paper:

```
int total = 0;

for (int i = 0; i < 3; i++) {
    total += i;
}
```

Write the value of `i` and `total` after every repetition, name the initialization / condition / update parts, and say why `i <= 3` would be wrong for a three-element array.

Checkpoint A - Copy and build the project

Start of the session - get your projects back from `H:`. `C:\Work` is cleaned on the lab PCs, and you may sit at a different PC than last week. If `C:\Work\<studentnumber>` does not contain `EE5203_L01_<studentnumber>`, copy **all** your project folders from `H:\EE_Labs` into `C:\Work\<studentnumber>` first. Do not run *Save to H* now: it makes `H:\EE_Labs` an exact copy of `C:\Work\<studentnumber>`, so running it on an empty folder deletes your saved projects. Do not build or debug directly on `H:`.

If your L01 project does not build (or it is not on `H:` either): download EE5203_L01_reference.zip, unzip it into `C:\Work\<studentnumber>`, and use the folder `EE5203_L01_reference` in step 1 instead of your L01 folder. Then continue with the same steps.

1. In `C:\Work\<studentnumber>`, copy the folder `EE5203_L01_<studentnumber>` and rename the copy `EE5203_L02_<studentnum` Check that `.vscode` came with it; if it is missing, copy it again from `C:\Users\Public\EE5203_template`.
2. In the new folder, **delete the `build` folder**. It still points to the L01 folder, and CMake stops with an error if you keep it.
3. Open `CMakeLists.txt` in the new folder. Change the project name to the new folder name:

```
# Set the project name
set(CMAKE_PROJECT_NAME EE5203_L02_<studentnumber>)
```

F5 looks for `build\Debug\<folder name>.elf`. If this line keeps the old name, the build works but F5 reports that the program file does not exist.

4. VS Code → File → Open Folder → `EE5203_L02_<studentnumber>`. Choose the **Debug** preset if asked. Build with **Ctrl+Shift+B**: zero errors. Read errors and warnings in the **Problems** panel (**Ctrl+Shift+M**), not in the terminal: click a row to jump to its line.
5. Find `USER CODE BEGIN 2` in `main.c`.

You do not need CubeMX in this lab.

Evidence: Show the title bar with `EE5203_L02_<studentnumber>` and the build line `Linking C executable EE5203_L02_<studentnumber>.elf`.

Checkpoint B - Declare the data and result variables

Inside `USER CODE BEGIN 2`, before the generated `while (1)`, add:

```
uint16_t samples[8] = {
    850, 1250, 4095, 4200,
    2050, 900, 3100, 2800
};

int sum = 0;
int average = 0;
int high_count = 0;
int invalid_count = 0;
int status_code = 0;
```

For each declaration, be able to name the type, the starting value, and the valid array indices.

Evidence: Build with zero errors and show the declarations.

Checkpoint C - Process every array element

Add this loop after the declarations:

```
for (int i = 0; i < 8; i++) {
    if (samples[i] > 4095) {
        samples[i] = 4095;
        invalid_count++;
    }

    sum += samples[i];

    if (samples[i] > 2500) {
        high_count++;
    }
}
```

Before running, answer:

1. Which input value will be changed?
2. What will replace it?
3. Which indices will the loop visit?
4. Why is `i <= 8` invalid?

Evidence: Break inside the loop, single-step two repetitions, and show `i`, `samples[i]`, `sum`, and `high_count`.

Checkpoint D - Calculate and classify the result

After the loop, add:

```
average = sum / 8;

if (average < 1500) {
    status_code = -1;
} else if (average > 3000) {
    status_code = 1;
} else {
    status_code = 0;
}
```

The numeric status is deliberate. C strings and character arrays are introduced in Lecture 3.

Before debugging, calculate the expected:

Result	Prediction
`samples[3]` after clamping	
`invalid_count`	
`high_count`	
`sum`	
`average`	

Result	Prediction
`status_code`	

Evidence: Break after the `if/else` statement and compare every observed value with the prediction.

Checkpoint E - Change the rule (your variant)

Your TA assigns one variant by seat. Each changes a **rule**, not a value, so the lecture numbers cannot be copied. Write your prediction **before** editing; the answer depends on a choice you must make and state.

Variant	Task	You must state
A	High now means above `2800`. Predict `high_count`; then predict it again for *at or above* `2800`.	why the two answers differ
B	Count **rising steps**: how many samples are larger than the sample before them?	whether you compare clamped or raw values, and which `i` you start at
C	Find the **index of the largest sample** with a `max_index` variable updated in the loop.	whether you used `>` or `>=`, and what changes with the other one
D	Find the **first index above** `4000` with a `while` loop and a compound condition (`i < 8 && …`). Then redo it for `4100`.	what the index means when nothing qualifies, and what the `i < 8` part protects

Evidence: the written prediction, the debugger value, and one sentence on any mismatch. Restore the original pipeline afterwards.

Stretch - make the board show the result (optional)

For students who finish early. First **delete the L01 blink** (the `HAL_GPIO_TogglePin`, `HAL_Delay(500)` and `toggle_count` lines in `USER CODE BEGIN 3`); otherwise the two blinks mix and cannot be counted. Then, inside the generated `while (1)`, blink LD2 `high_count` times, wait one second, and repeat:

```
for (int k = 0; k < high_count; k++) {
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    HAL_Delay(150);
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    HAL_Delay(150);
}
HAL_Delay(1000);
```

Count the blinks. They must agree with `high_count` in the debugger — two independent pieces of evidence for one number. Then let the status choose the speed: set an `int period` to `100` for HIGH and `500` otherwise with an `if/else`, and use it in both delays. Show the TA both versions.

Common problems

Symptom	First check
Unknown type at build	`#include <stdint.h>` is present
Result too large / loop never stops	`sum` starts at zero; `i++` is present
Memory changes unexpectedly	condition is `i < 8`, not `i <= 8`
Change has no effect	rebuild and restart the debug session
Warnings disappeared, but nothing was fixed	only changed files are compiled again: save `main.c` and build again
`EE5203_L01_<studentnumber>` is not in `C:\Work`	copy your projects from `H:\EE_Labs` first (start of Checkpoint A); never run *Save to H* on an empty folder
`CMake Error: The current CMakeCache.txt directory ... is different`	delete the copied `build` folder (Checkpoint A, step 2)
F5: program file `...EE5203_L02_<studentnumber>.elf` does not exist	the `set(CMAKE_PROJECT_NAME ...)` line matches the folder name (step 3)

Acceptance checklist

- ☐ Project builds with zero errors.
- ☐ Only `main.c` was edited.
- ☐ Array indices stay between `0` and `7`.
- ☐ `4200` is replaced by `4095`.
- ☐ Sum, average, counts, and status match the prediction.
- ☐ Your Checkpoint E variant is predicted, demonstrated and explained.
- ☐ (Stretch) blink count on the board equals `high_count` in the debugger.
- ☐ Every submitted screenshot shows readable variables.
- ☐ *Save to H* has run and `H:\EE_Labs\EE5203_L02_<studentnumber>` exists.

Individual oral check

Be prepared to answer one question and make one live change:

- What is the difference between `=` and `==`?
- Why does the loop use `i < 8`?
- What value does `samples[i]` select during a repetition?
- Why must `sum` start at zero?
- Change one threshold and predict the result before running.

End of the session - Save to H

1. Close VS Code.
2. Run the desktop **Save to H** shortcut. It copies `C:\Work\<studentnumber>` (without the `build` folders) to `H:\EE_Labs`.
3. Check that `H:\EE_Labs\EE5203_L02_<studentnumber>` exists before you log out. You need this project next week.

Submit

Submit `main.c`, the completed prediction table, one build-console screenshot, one debugger screenshot of the final array and results, and three sentences explaining your Checkpoint E variant.

Marks (10): pre-lab quiz on CATS 1 · A copy and build 1 · B declarations 1 · C processing loop 2 · D calculate and classify 2 · E rule change 1 · oral check 2.